

Part 1: QA Strategy and Release Readiness

Definition of Done:

To consider a feature done, QA must ensure a mentor is comfortable using it deep in the village with only one bar of signal.

- I must test the USSD flow end-to-end on real SIM cards across all networks; we should not only assume with a simulator, every confirmation also carries a unique 6-digit ID
- I must ensure the offline flow is proven. Ensure I have tested airplane mode mid-registration, a page refresh while offline, closing the browser, multiple queued records, and a flaky connection that drops and returns repeatedly. Everything syncs once, and nothing is lost or doubled.
- I must ensure it works on the devices mentors actually use. I have tested on low-end Android phones, older browsers, and slow 2G/3G connections.
- I must ensure every acceptance criteria listed has a valid test case, including unhappy paths, and all of them pass before pushing to production.
- I must ensure validation works perfectly as expected. Ensure validations with blank and non-numeric input are handled. Phone numbers are validated for country formats across all telecom networks.
- Ensure that real users have tried; I ensure we do a UAT with a manual or guide for this feature

Two Potential risks I would flag

- Offline sync creating duplicates or losing data. If the sync retries after a timeout, the server may have already saved the record, and now we may have two. If we have a duplicate, this is then a failure. If the browser cache gets cleared, or a phone is shared between mentors, queued records can easily disappear. To solve this, I would advise using client-generated unique IDs with idempotent sync on the server, plus a visible X number of records waiting to sync warning so mentors know what's pending.
- USSD session timeouts and SMS delivery failures. USSD sessions time out quickly, so it could drop off mid-flow. That leaves questions like: is a half-finished registration saved? Does the SMS go out for a record that never committed? Separately, SMS delivery depends on the telecom network, so messages can be delayed or silently fail. This causes one to walk away without their ID and might register again. I would also advise checking how the 6-digit IDs are generated so we never get collisions.

Bug Triage

Bug A: On low-end mobile browsers, the district dropdown menu renders off-screen.

Severity: HIGH, Priority: Must be fixed before release

- This looks like just a UI bug, but district is a required field. If mentors can't select it, they can't register anyone. And rural mentors are exactly the people most likely to be on low-end phones. So it blocks a core flow for a big chunk of our real users. USSD is a possible workaround, but it's not a fix.

Bug B: When a mentor loses connection mid-registration, duplicate participant records are created upon re-establishing connection.

Severity: Critical, Priority: Must be fixed before release,

- This bug is the most critical one. It corrupts data silently, and nobody notices until the attendance numbers and impact reports are wrong. We report those numbers to funders, and cleaning duplicates out of production data is painful. Worse, it happens in exactly the scenario offline mode exists for. This is a release blocker.

Bug C: The confirmation SMS takes up to 4 minutes to arrive instead of being Instant.

Severity: Medium, Priority: High on the list but not a release blocker

- This doesn't break registration, because the record still saves. But it still matters, though. The sms should arrive immediately, and one waiting 4 minutes will probably assume it failed and try again, which feeds right back into duplicates. I would investigate whether the delay comes from our message queue or the SMS aggregator, since the fix looks very different depending on the answer. If it turns out to be telecom network side, I'd consider showing the ID on the USSD screen itself as a fallback.

Part 2: Code Test Hands-On Automation & API Testing

Link to github:

<https://github.com/akram256/EducateInterview/tree/main/attendance-api-tests>

Part 3: CI/CD Pipeline and E2E Tooling

1. Framework Trade-off: Playwright vs Cypress

I have had a chance to use Selenium, Playwright, and Cypress during my career as a QA engineer.

Cypress is the easiest place to start. It's beginner-friendly and has great debugging and a big community. But it runs inside the browser, so multi-tab flows get messy, Safari support is still experimental, and fast parallel runs in CI usually mean paying for Cypress Cloud, which is an extra cost I'd rather a new team avoid. Selenium supports everything, but it's slower and flakier, and keeping it stable takes a lot of maintenance for a small team.

That's why I would go with Playwright. It waits for elements automatically, so tests break less often. It covers Chrome, Firefox, and Safari's engine out of the box, and it can emulate low-end Android phones, which is what our field mentors actually use. It can also simulate going offline, which is exactly what we need to test offline sync. Parallel runs are free, so the suite stays fast without adding cost. It's a bit harder to learn than Cypress, but the lower maintenance makes that worth it.

2. CI/CD Pipeline(GitHub Actions)

```
name: Educate Quality Gate

on:
  pull_request:
    branches: [main]

# This cancels older runs when new commits are pushed to the same PR
concurrency:
  group: ${ github.workflow }-${ github.ref }
  cancel-in-progress: true

jobs:
  test:
    name: Lint, Unit & E2E Tests
    runs-on: ubuntu-latest
    timeout-minutes: 20

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm

      - name: Install dependencies
        run: npm ci

      - name: Lint
        run: npm run lint

      - name: Unit tests
        run: npm test

      - name: Install Playwright browsers
        run: npx playwright install --with-deps

      # We can run desktop + mobile projects defined in playwright.config.ts
      # (e.g. Desktop Chrome, Pixel 5, Mobile Safari)
      - name: Run E2E tests
        run: npx playwright test
        env:
```

```
BASE_URL: ${ secrets.STAGING_URL }
```

```
- name: Upload test report  
  if: failure()  
  uses: actions/upload-artifact@v4  
  with:  
    name: playwright-report  
    path: playwright-report/  
    retention-days: 7
```